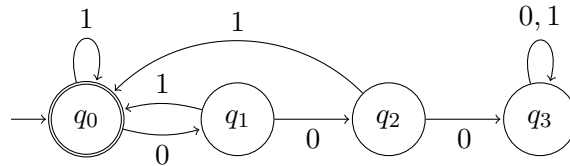


# Computability - 2020 Moed A Solution

## Question 1

A minimal DFA for  $L$  is shown below.



We show that  $L$  has at least 4 Myhill-Nerode equivalence classes, and since the DFA above has 4 states, its minimality follows by the Myhill-Nerode theorem.

The following words are pairwise nonequivalent:  $\epsilon$ , 0, 00, 000.  $\epsilon$  (which is in the language) is separated from the other three words using  $\epsilon$ , since they end with 0 which isn't followed by 1 or 01. 0 and 00 are separated from each other by 01, since  $001 \in L$  and  $0001 \notin L$ . 0 and 000 are separated by 01, since  $001 \in L$  and  $00001 \notin L$ . Finally, 00 and 000 are separated by 1, since  $001 \in L$  and  $0001 \notin L$ . We conclude that each of these four words has its own equivalence class, and the result follows.

## Question 2

The following grammar generates  $L$ :

$$\begin{aligned} S &\rightarrow TaT \\ T &\rightarrow aTbT|bTaT|\epsilon \end{aligned}$$

Other grammars for  $L$  exist.

## Question 3

We start by showing a simple lemma:

**Lemma:**  $L$  is simple iff the minimal DFA for  $L$  has at most one accepting state.

**Proof:**

- If the minimal DFA for  $L$  has a single accepting state then  $L$  is simple by definition.
- If the minimal DFA for  $L$  has no accepting states (and so  $L = \emptyset$ ), then we can add a single unreachable accepting state. Thus,  $L$  is simple.
- If  $L$  is simple, then there is a DFA  $\mathcal{A}$  with a single accepting state  $q$  such that  $L(\mathcal{A}) = L$ . When we apply the minimization algorithm on  $\mathcal{A}$ , we get a DFA  $\mathcal{B}$ , whose set of states corresponds to a partition of the states of  $\mathcal{A}$  to sets. In particular, if  $q$  is reachable, it appears in a single accepting state in  $\mathcal{B}$ , corresponding to the singleton  $\{q\}$ , while all other states in  $\mathcal{B}$  are not accepting. Thus, the minimal DFA for  $L$ ,  $\mathcal{B}$ , has a single accepting state. If  $q$  is unreachable, there are no accepting states in  $\mathcal{B}$ .

Given this lemma, the polynomial algorithm works as follows: given a DFA  $\mathcal{A}$ , find the minimal equivalent DFA  $\mathcal{B}$ , and return "Yes" iff  $\mathcal{B}$  has at most one accepting state. We have seen in class that we can find  $\mathcal{B}$  in polynomial time, and so our algorithm is polynomial as well. Correctness follows from the lemma.

## Question 4

The claim is **true**.

Since  $L \in R$  there exists a TM  $M$  that decides  $L$ . We use  $M$  to construct a TM  $M'$  which decides  $Mark(L)$ .

**Construction:** Given an input  $x$ ,  $M'$  works as follows:

1. If  $x$  is not in the format  $b\#w$  for  $b \in \{0, 1\}$  and  $w \in \Sigma^*$ , reject.
2. Run  $M$  on  $w$ . If ( $M$  accepts and  $b = 1$ ) or ( $M$  rejects and  $b = 0$ ) - accept. Otherwise - reject.

**Correctness:** We prove that  $L(M') = Mark(L)$ . Well,  $x \in L(M')$  iff  $M'$  accepts  $x$  iff ( $x$  is in the correct format, i.e.  $x = b\#w$ ) and ( $M$  accepts  $w$  and  $b = 1$ ) or ( $M$  rejects  $w$  and  $b = 0$ ) iff ( $x$  is in the correct format, i.e.  $x = b\#w$ ) and ( $w \in L$  and  $b = 1$ ) or ( $w \notin L$  and  $b = 0$ ) iff  $x \in Mark(L)$ . Note that the third iff follows from the fact that  $M$  decides  $L$ .

**Computability:** Checking for the correct format is easy (it is a regular language so it is easily done by a TM). Next, running  $M$  on  $w$  can be done using a universal machine. Since  $M$  is a decider, it always halts and so  $M'$  also always halts.

**Alternative Solutions:**

- Use closure properties. The languages  $\{0\#\}$  and  $\{1\#\}$  are both finite so they are regular and in particular in  $R$ . The languages  $L$  and  $\bar{L}$  are both in  $R$ . As  $R$  is closed for concatenation we get that  $\{1\#\} \cdot L$  and  $\{0\#\} \cdot \bar{L}$  are in  $R$ . Finally, from closure to union we get that the language  $\{0\#\} \cdot \bar{L} \cup \{1\#\} \cdot L = Mark(L)$  is in  $R$ , as required.

## Question 5

The claim is **false**.

**Proof:** Assume (by contradiction) that the claim is true, which means that  $L \in RE$  implies  $Mark(L) \in RE$ . From question 6 we know that  $Mark(L) \in RE$  implies  $L \in R$ , so we get that  $L \in RE$  implies  $L \in R$ , and in other words  $RE \subseteq R$  which is clearly false (for example, we saw languages in  $RE \setminus R$ , like  $A_{TM}$ ,  $HALT_{TM}$ , etc).

**Alternative Solutions:**

- Take a language  $L \in RE \setminus R$  (for example  $A_{TM}$ ) and show that if  $Mark(L)$  is in  $RE$  we can decide  $L$  (exactly like in the solution to question 6), contradicting  $L \notin R$ . One can also show how to simply recognize  $\bar{L}$  (given input  $w$  simulate the TM that recognizes  $Mark(L)$  on  $0\#w$  and answers the same), contradicting  $L \notin coRE$
- Take a language  $L \in RE \setminus R$  (for example  $A_{TM}$ ) and show a reduction  $\bar{L} \leq_m Mark(L)$  by  $x \mapsto 0\#x$ . By the definition of  $Mark(L)$ , we know that  $x \in \bar{L}$  iff  $0\#x \in Mark(L)$ . Since  $L \notin coRE$ , it follows that  $\bar{L} \notin RE$ , and from the reduction theorem, we get that  $Mark(L) \notin RE$ .

## Question 6

The claim is **true**.

Since  $Mark(L) \in RE$ , there exists a TM  $M$  which recognizes  $Mark(L)$ . We use  $M$  to construct a TM  $M'$  which decides  $L$ .

**Construction:** Given an input  $w$ ,  $M'$  works as follows:

1. For  $i = 1, 2, 3, \dots$ :

- (a) Simulate  $M$  on  $x_0 = 0\#w$  for at most  $i$  steps. If it accepts - reject.
- (b) Simulate  $M$  on  $x_1 = 1\#w$  for at most  $i$  steps. If it accepts - accept.

**Correctness:** We prove that  $L(M') = L$ .

- $L(M') \supseteq L$

Assume  $w \in L$ . Since  $L(M) = Mark(L)$  and  $1\#w = x_1 \in Mark(L)$ , the TM  $M$  accepts  $x_1$ . Denote by  $i_1$  the number of steps it takes  $M$  to accept  $x_1$ . When  $i$  reaches  $i_1$  the TM  $M'$  simulates  $M$  on  $x_1$  for  $i_1$  steps.  $M'$  finds out  $M$  accepts, and so  $M'$  accepts. Note that it cannot be the case that  $M'$  halts and rejects at stage 1a (before reaching iteration  $i = i_1$ ), because  $w \in L$  and therefore  $0\#w \notin Mark(L)$ , so  $M$  does not accept  $0\#w$ .

- $L(M') \subseteq L$

Assume  $w \notin L$ . Since  $L(M) = Mark(L)$  and  $0\#w = x_0 \in Mark(L)$ , the TM  $M$  accepts  $x_0$ . Denote by  $i_0$  the number of steps it takes  $M$  to accept  $x_0$ . When  $i$  reaches  $i_0$  the TM  $M'$  simulates  $M$  on  $x_0$  for  $i_0$  steps.  $M'$  finds out that  $M$  accepts, and so  $M'$  rejects. Note that it cannot be the case that  $M'$  halts and accepts at stage 1b (before reaching iteration  $i = i_0$ ), because  $w \notin L$  and therefore  $1\#w \notin Mark(L)$  so  $M$  does not accept  $1\#w$ .

**Computability:** The TM  $M'$  can be implemented using a universal machine (in fact, we saw in the course how to run two TMs in parallel, and this is exactly what  $M'$  does).

$M'$  always halts, since for every word  $w \in \Sigma^*$  it holds that  $w \in L$  or  $w \notin L$ . In the first case  $x_1 = 1\#w$  is in  $Mark(L)$  and  $M$  eventually accepts, so  $M$  halts (and accepts). In the second case  $x_0 = 0\#w$  is in  $Mark(L)$  and  $M$  eventually accepts, so  $M$  halts (and rejects).

**Alternative Solutions:**

- Showing two reductions:  $L \leq_m Mark(L)$  by  $x \mapsto 1\#x$  and  $\bar{L} \leq_m Mark(L)$  by  $x \mapsto 0\#x$ . Since  $Mark(L) \in RE$ , we get (from the reduction theorem) that  $L \in RE$  and  $\bar{L} \in RE$ , so  $L \in RE \cap coRE = R$ .

## Question 7

The claim is **true**.

**Proof 1:** It is known that PATH is decidable in P, and that  $P \subseteq NP$ . Hence, PATH is in NP. We have seen that HAMCYCLE is NP-Hard. It follows that  $L \leq_P HAMCYCLE$  for all  $L \in NP$ , and in particular, for  $L = PATH$ .

**Proof 2:** Let  $G_0$  be a graph with no Hamiltonian cycle, and  $G_1$  be a graph with a Hamiltonian cycle. Given input  $G$  to PATH, the reduction checks if  $G$  is in PATH. If it is, the reduction outputs  $G_1$ . Otherwise, it outputs  $G_0$ . Since PATH is decidable in P, the reduction is indeed polynomial.

## Question 8

The claim is **unknown**.

If it is true, the following claims are also true:  $P = NP$ ,  $NP = \text{coNP}$  and  $P = \text{PSPACE}$ .

**Proof:** We have seen that TQBF is PSPACE-hard. Thus, for every  $L$  in PSPACE we have  $L \leq_P \text{TQBF}$ . If TQBF is decidable in  $P$ , we can, by the reduction theorem, decide every language in PSPACE in polynomial time. In addition, since both  $NP$  and  $\text{coNP}$  are subsets of PSPACE and supersets of  $P$ , it follows that  $P = NP = \text{coNP}$ .

## Question 9

The claim is **unknown**.

If it is true, the following claims are also true:  $P = NP$  and  $NP = \text{coNP}$ .

**Proof:** We have seen that CLIQUE is NP-Hard. Thus, for all  $L$  in  $NP$ , we have that  $L \leq_P \text{CLIQUE}$ . In addition, PATH is decidable in  $P$ . Therefore, if  $\text{CLIQUE} \leq_P \text{PATH}$ , by the reduction theorem we get that  $P = NP$ . In addition, since  $P$  is closed under complementation, we get that  $P = \text{coNP}$  and so  $NP = \text{coNP}$ .

Note: PATH is in  $NL$ , but the reduction is polynomial, and so we cannot conclude that  $NL = NP$ .

## Question 10

The language MCLIQUE is NP-Complete:

- The language is in  $NP$  since there is a poly-time verifier: The verifier takes  $\langle G, k, m, S, T \rangle$ . It verifies that:
  - $G$  is a graph
  - $m$  is greater than 0 and less than  $k$
  - $S$  and  $T$  are sets of vertices of size  $k$  each
  - The size of the intersection of  $S$  and  $T$  is  $m$
  - $S$  and  $T$  are both cliques in  $G$

All of these checks can be done in poly-time (the intersection is done in  $O((|S| + |T|)^2)$  and we have seen that checking that a set is a clique can be done in poly-time).

- The language is NP-Hard since we can show a reduction from CLIQUE. We have seen that CLIQUE is NP-Hard and from the reduction theorem it follows that MCLIQUE is also NP-Hard. The reduction takes  $\langle G, k \rangle$  and outputs  $\langle G', k+1, 1 \rangle$  where  $G'$  is a graph constructed as follows: Duplicate  $G$  into two identical copies that are disjoint. Then add a new vertex  $x$ , and connect it to every vertex in both copies of  $G$ . We claim that  $\langle G, k \rangle \in \text{CLIQUE}$  iff  $\langle G', k+1, 1 \rangle \in \text{MCLIQUE}$ :
  - If  $\langle G, k \rangle$  is in CLIQUE there exists a clique of size  $k$  in  $G$ . This means that there are two disjoint  $k$ -cliques in  $G'$  - one in each of the separate copies of  $G$ . Each of these cliques is a  $(k+1)$ -clique when adding  $x$ , and they clearly have an intersection of size 1 - they only intersect on  $x$ .
  - If  $\langle G, k \rangle$  is not in CLIQUE, there does not exist a  $k$ -clique in  $G$ . Therefore, there does not exist a  $k$ -clique in either of the disjoint copies of  $G$  in  $G'$ . Since there is no  $k$ -clique in either of the copies, there cannot be a  $(k+1)$ -clique in  $G'$ , even when adding the vertex  $x$ , since there are no edges between the distinct copies of  $G$  in  $G'$ .